

Hai con trỏ (Two Pointers)

Nội dung

Chào mừng bạn đến với bài học về kỹ thuật Hai con trỏ (Two Pointers)! Đây là một trong những kỹ thuật quan trọng nhất trong lập trình thi đấu, giúp bạn giải quyết rất nhiều bài toán về đoạn con và cặp phần tử một cách hiệu quả.

1. Tại sao cần kỹ thuật hai con trỏ?
2. Ý tưởng cốt lõi
3. Hai con trỏ cùng chiều (Sliding Window)
 - Tính chất quan trọng (tính đơn điệu)
 - Template cơ bản
 - Ví dụ chi tiết: Đoạn con dài nhất có tổng $\leq S$
 - Tại sao chỉ $O(n)$?
 - Ví dụ 2: Đoạn con dài nhất không có phần tử trùng
4. Hai con trỏ ngược chiều
 - Ý tưởng
 - Ví dụ chi tiết: Two Sum trên mảng đã sắp xếp
 - Tại sao cách này đúng? (Rất quan trọng!)
 - Code hoàn chỉnh
5. Trộn hai mảng đã sắp xếp (Merge)
 - Ý tưởng
 - Ví dụ chi tiết
 - Code hoàn chỉnh
6. Bài toán mẫu hoàn chỉnh: Ferris Wheel (CSES 1090)
7. Sai lầm thường gặp và mẹo
 - Sai lầm 1: Quên xử lý cửa sổ rỗng
 - Sai lầm 2: Không duy trì bất biến (invariant)
 - Sai lầm 3: Dùng two pointers khi không có tính đơn điệu
 - Mẹo 1: Đếm thay vì tìm max
 - Mẹo 2: Dùng map/set cho điều kiện phức tạp
8. Tổng kết

1. Tại sao cần kỹ thuật hai con trỏ?

Hãy bắt đầu với một câu hỏi đơn giản:

Bài toán: Cho mảng a gồm n số nguyên dương và số S . Tìm đoạn con liên tiếp dài nhất có tổng $\leq S$.

Cách "ngây thơ" (brute force) — $O(n^2)$: Thử tất cả các cặp (l, r) , tính tổng đoạn $[l, r]$, kiểm tra xem $\leq S$ không:

C++

```
int ans = 0;
for (int l = 0; l < n; l++) {
    long long sum = 0;
    for (int r = l; r < n; r++) {
        sum += a[r];
        if (sum <= S) ans = max(ans, r - l + 1);
        else break;
    }
}
```

Python

```
ans = 0
for l in range(n):
    total = 0
    for r in range(l, n):
        total += a[r]
        if total <= S:
            ans = max(ans, r - l + 1)
        else:
            break
```

Với $n = 10^5$, cách này cần khoảng 10^{10} phép tính — quá chậm! (nhớ rằng máy tính chỉ xử lý khoảng 10^8 phép tính mỗi giây).

Câu hỏi đặt ra: Liệu có cách nào chỉ cần $O(n)$ — tức là duyệt mảng một lần — mà vẫn tìm được đáp án?

Câu trả lời là CÓ, nhờ kỹ thuật hai con trỏ!

2. Ý tưởng cốt lõi

Kỹ thuật hai con trỏ dùng hai biến (thường gọi là l và r) để đại diện cho hai "vị trí" trên mảng. Thay vì thử tất cả các cặp (l, r) , ta tận dụng tính đơn điệu của bài toán để di chuyển l và r một cách thông minh.

Hình ảnh trực quan: Hãy tưởng tượng bạn đang nhìn qua một cửa sổ trượt (sliding window) trên mảng. Cửa sổ này có thể mở rộng (kéo r sang phải) hoặc thu hẹp (kéo l sang phải). Bạn không bao giờ kéo sang trái — cửa sổ chỉ "trượt" về phía trước.

Ví dụ đời thường: Bạn đang đọc một trang sách dài. Bạn dùng hai ngón tay đánh dấu đoạn đang đọc: ngón trái (l) đánh dấu đầu đoạn, ngón phải (r) đánh dấu cuối đoạn. Khi muốn đọc thêm, bạn kéo ngón phải sang phải. Khi đoạn quá dài, bạn kéo ngón trái sang phải. Cả hai ngón tay đều chỉ di chuyển sang phải — không bao giờ quay lại.

Có ba dạng phổ biến:

- **Hai con trỏ cùng chiều (Sliding Window):** l và r cùng đi từ trái sang phải
- **Hai con trỏ ngược chiều:** l đi từ trái, r đi từ phải, gặp nhau ở giữa
- **Trộn hai mảng đã sắp xếp:** hai con trỏ trên hai mảng khác nhau

3. Hai con trỏ cùng chiều (Sliding Window)

Đây là dạng phổ biến nhất. Dùng khi cần tìm đoạn con thỏa mãn một điều kiện nào đó.

Tính chất quan trọng (tính đơn điệu)

Kỹ thuật này hoạt động khi bài toán có tính chất sau:

"Nếu đoạn $[l, r]$ không thỏa mãn điều kiện, thì đoạn $[l, r + 1], [l, r + 2], \dots$ cũng không thỏa mãn."

Nói cách khác: khi cửa sổ $[l, r]$ đã vi phạm, mở rộng thêm chỉ làm tệ hơn. Ta phải thu hẹp cửa sổ (tăng l) để có thể thỏa mãn lại.

Template cơ bản

C++

```
int l = 0;
for (int r = 0; r < n; r++) {
    // Bước 1: Thêm a[r] vào cửa sổ [l, r]
    // (cập nhật biến theo dõi: tổng, đếm, map, ...)
    // Bước 2: Thu hẹp cửa sổ nếu vi phạm
    while (/* cửa sổ [l, r] vi phạm điều kiện */) {
        // Loại a[l] khỏi cửa sổ
        l++;
    }
    // Bước 3: Cập nhật đáp án
    // Đoạn [l, r] giờ thỏa mãn điều kiện
    ans = max(ans, r - l + 1); // hoặc ans += r - l + 1, tùy bài
}
```

Python

```
l = 0
for r in range(n):
    # Bước 1: Thêm a[r] vào cửa sổ [l, r]
    # (cập nhật biến theo dõi: tổng, đếm, dict, ...)
    # Bước 2: Thu hẹp cửa sổ nếu vi phạm
    while False: # thay bằng điều kiện vi phạm
```

```

    # Loại a[l] khỏi cửa sổ
    l += 1
# Bước 3: Cập nhật đáp án
# Đoạn [l, r] giờ thỏa mãn điều kiện
ans = max(ans, r - l + 1) # hoặc ans += r - l + 1, tùy bài

```

Ví dụ chi tiết: Đoạn con dài nhất có tổng $\leq S$

Đề bài: Cho mảng $a = [2, 1, 3, 5, 1, 2]$ và $S = 7$. Tìm đoạn con liên tiếp dài nhất có tổng ≤ 7 .

Hãy mô phỏng từng bước (chỉ số bắt đầu từ 0):

BƯỚC	r	$a[r]$	THÊM	sum	VI PHẠM?	ĐOẠN $[l, r]$	ĐỘ DÀI
1	0	$a[0] = 2$	2	2	$\leq 7 \checkmark$	$[0, 0]$	1
2	1	$a[1] = 1$	1	3	$\leq 7 \checkmark$	$[0, 1]$	2
3	2	$a[2] = 3$	3	6	$\leq 7 \checkmark$	$[0, 2]$	3
4	3	$a[3] = 5$	5	11	$11 > 7 \times$: l tăng $0 \rightarrow 1$ ($sum = 9$), $1 \rightarrow 2$ ($sum = 8$), $2 \rightarrow 3$ ($sum = 5$)	$[3, 3]$	1
5	4	$a[4] = 1$	1	6	$\leq 7 \checkmark$	$[3, 4]$	2
6	5	$a[5] = 2$	2	8	$8 > 7 \times$: l tăng $3 \rightarrow 4$ ($sum = 3$)	$[4, 5]$	2

Kết quả: Đoạn dài nhất là $[0, 2] = \{2, 1, 3\}$ có tổng 6, độ dài 3.

Code hoàn chỉnh:

C++

```

#include <bits/stdc++.h>
using namespace std;
int main() {
    int n;
    long long S;
    cin >> n >> S;
    vector<int> a(n);
    for (int &x : a) cin >> x;
    long long sum = 0;
    int l = 0, ans = 0;
    for (int r = 0; r < n; r++) {
        sum += a[r]; // Thêm a[r] vào cửa sổ
        while (sum > S) { // Nếu tổng vượt quá S
            sum -= a[l]; // Loại a[l] ra
            l++; // Thu hẹp cửa sổ
        }
        ans = max(ans, r - l + 1); // Cập nhật đáp án
    }
    cout << ans << endl;
}

```

```
}
```

Python

```
n, S = map(int, input().split())
a = list(map(int, input().split()))
total = 0
l = 0
ans = 0
for r in range(n):
    total += a[r] # Thêm a[r] vào cửa sổ
    while total > S: # Nếu tổng vượt quá S
        total -= a[l] # Loại a[l] ra
        l += 1 # Thu hẹp cửa sổ
    ans = max(ans, r - l + 1) # Cập nhật đáp án
print(ans)
```

Tại sao chỉ $O(n)$?

Nhiều bạn lo lắng: "Có vòng `while` bên trong vòng `for`, không phải $O(n^2)$ sao?"

Không! Hãy nhìn biến l : nó chỉ tăng, không bao giờ giảm. Trong suốt quá trình:

- r đi từ 0 đến $n - 1$: tổng cộng n bước
- l đi từ 0 đến tối đa $n - 1$: tổng cộng tối đa n bước

Vậy tổng số phép tính là $n + n = 2n = O(n)$. Mỗi phần tử được thêm vào cửa sổ đúng 1 lần và bị loại ra đúng 1 lần.

Ví dụ 2: Đoạn con dài nhất không có phần tử trùng

Đề bài: Cho mảng $a = [1, 3, 2, 3, 4, 1]$. Tìm đoạn con liên tiếp dài nhất mà tất cả các phần tử đều khác nhau.

Ý tưởng: Dùng set hoặc map để theo dõi các phần tử trong cửa sổ hiện tại. Khi thêm $a[r]$ vào mà nó đã có trong cửa sổ, thu hẹp l cho đến khi loại bỏ phần tử trùng.

Mô phỏng:

r	$a[r]$	CỬA SỔ TRƯỚC	TRÙNG?	SAU KHI LOẠI	ĐOẠN	DÀI
0	1	$\emptyset$	Không	$\{1\}$	$[0, 0]$	1
1	3	$\{1\}$	Không	$\{1, 3\}$	$[0, 1]$	2
2	2	$\{1, 3\}$	Không	$\{1, 3, 2\}$	$[0, 2]$	3
3	3	$\{1, 3, 2\}$	Có (3!)	Loại 1, loại 3 $\rightarrow \{2, 3\}$	$[2, 3]$	2
4	4	$\{2, 3\}$	Không	$\{2, 3, 4\}$	$[2, 4]$	3
5	1	$\{2, 3, 4\}$	Không	$\{2, 3, 4, 1\}$	$[2, 5]$	4

Đáp án: Đoạn $[2, 5] = \{2, 3, 4, 1\}$, dài 4.

C++

```

#include <bits/stdc++.h>
using namespace std;
int main() {
    int n;
    cin >> n;
    vector<int> a(n);
    for (int &x : a) cin >> x;
    set<int> window;
    int l = 0, ans = 0;
    for (int r = 0; r < n; r++) {
        while (window.count(a[r])) {
            window.erase(a[l]);
            l++;
        }
        window.insert(a[r]);
        ans = max(ans, r - l + 1);
    }
    cout << ans << endl;
}

```

Python

```

n = int(input())
a = list(map(int, input().split()))
window = set()
l = 0
ans = 0
for r in range(n):
    while a[r] in window:
        window.remove(a[l])
        l += 1
    window.add(a[r])
    ans = max(ans, r - l + 1)
print(ans)

```

4. Hai con trỏ ngược chiều

Dùng trên mảng đã sắp xếp để tìm cặp phần tử thỏa mãn điều kiện.

Ý tưởng

Đặt con trỏ l ở đầu mảng (phần tử nhỏ nhất) và r ở cuối mảng (phần tử lớn nhất). Dựa vào kết quả so sánh, quyết định di chuyển l sang phải (tăng tổng) hoặc r sang trái (giảm tổng).

Ví dụ chi tiết: Two Sum trên mảng đã sắp xếp

Đề bài: Cho mảng đã sắp xếp $a = [1, 3, 5, 7, 10, 12]$ và $x = 13$. Tìm hai số có tổng bằng x .

Mô phỏng:

- Bước 1: $l = 0, r = 5 \rightarrow a[0] + a[5] = 1 + 12 = 13 = x \rightarrow$ TÌM THẤY! ✓

Ở ví dụ này ta may mắn tìm thấy ngay. Hãy thử với $x = 8$:

- Bước 1: $l = 0, r = 5 \rightarrow a[0] + a[5] = 1 + 12 = 13 > 8 \rightarrow$ giảm r
- Bước 2: $l = 0, r = 4 \rightarrow a[0] + a[4] = 1 + 10 = 11 > 8 \rightarrow$ giảm r
- Bước 3: $l = 0, r = 3 \rightarrow a[0] + a[3] = 1 + 7 = 8 = x \rightarrow$ TÌM THẤY! ✓

Thử với $x = 20$:

- Bước 1: $l = 0, r = 5 \rightarrow 1 + 12 = 13 < 20 \rightarrow$ tăng l
- Bước 2: $l = 1, r = 5 \rightarrow 3 + 12 = 15 < 20 \rightarrow$ tăng l
- Bước 3: $l = 2, r = 5 \rightarrow 5 + 12 = 17 < 20 \rightarrow$ tăng l
- Bước 4: $l = 3, r = 5 \rightarrow 7 + 12 = 19 < 20 \rightarrow$ tăng l
- Bước 5: $l = 4, r = 5 \rightarrow 10 + 12 = 22 > 20 \rightarrow$ giảm r
- Bước 6: $l = 4, r = 4 \rightarrow l \geq r \rightarrow$ KHÔNG TÌM THẤY ✕

Tại sao cách này đúng? (Rất quan trọng!)

Giả sử mảng đã sort tăng dần. Ta cần tìm $a[i] + a[j] = x$ với $i < j$.

- Nếu $a[l] + a[r] = x$: tìm thấy!
- Nếu $a[l] + a[r] < x$: ta cần tổng lớn hơn. Vì $a[r]$ đã là số lớn nhất ta đang xét, cách duy nhất là tăng $a[l] \rightarrow$ tăng l . Lúc này, ta loại bỏ $a[l]$ — nhưng $a[l]$ kết hợp với bất kỳ $a[j]$ nào ($j \leq r$) đều cho tổng $\leq a[l] + a[r] < x$, nên $a[l]$ không thể là phần tử trái trong đáp án.
- Nếu $a[l] + a[r] > x$: tương tự, giảm r . $a[r]$ kết hợp với bất kỳ $a[i]$ nào ($i \geq l$) đều cho tổng $\geq a[l] + a[r] > x$.

Mỗi bước loại bỏ đúng một phần tử mà chắc chắn không nằm trong đáp án $\rightarrow$ ta không bỏ sót nghiệm!

Code hoàn chỉnh

C++

```
#include <bits/stdc++.h>
using namespace std;
int main() {
    int n, x;
    cin >> n >> x;
    vector<int> a(n);
    for (int &v : a) cin >> v;
    sort(a.begin(), a.end());
    int l = 0, r = n - 1;
    while (l < r) {
```

```

    int s = a[l] + a[r];
    if (s == x) {
        cout << a[l] << " " << a[r] << endl;
        break;
    } else if (s < x) {
        l++;
    } else {
        r--;
    }
}
if (l >= r) cout << "IMPOSSIBLE" << endl;
}

```

Python

```

n, x = map(int, input().split())
a = list(map(int, input().split()))
a.sort()
l, r = 0, n - 1
found = False
while l < r:
    s = a[l] + a[r]
    if s == x:
        print(a[l], a[r])
        found = True
        break
    elif s < x:
        l += 1
    else:
        r -= 1
if not found:
    print("IMPOSSIBLE")

```

5. Trộn hai mảng đã sắp xếp (Merge)

Đây là thao tác nền tảng trong Merge Sort và cũng là dạng hai con trỏ.

Ý tưởng

Cho hai mảng đã sắp xếp A và B . Dùng hai con trỏ i (trên A) và j (trên B). Tại mỗi bước, so sánh $A[i]$ và $B[j]$, lấy phần tử nhỏ hơn đưa vào kết quả.

Ví dụ chi tiết

$A = [1, 4, 7, 9]$, $B = [2, 3, 5, 8]$

- Bước 1: $i = 0, j = 0 \rightarrow A[0] = 1 < B[0] = 2 \rightarrow$ lấy 1, $i++ \rightarrow result = [1]$
- Bước 2: $i = 1, j = 0 \rightarrow A[1] = 4 > B[0] = 2 \rightarrow$ lấy 2, $j++ \rightarrow result = [1, 2]$

- Bước 3: $i = 1, j = 1 \rightarrow A[1] = 4 > B[1] = 3 \rightarrow$ lấy 3, $j++ \rightarrow result = [1, 2, 3]$
- Bước 4: $i = 1, j = 2 \rightarrow A[1] = 4 < B[2] = 5 \rightarrow$ lấy 4, $i++ \rightarrow result = [1, 2, 3, 4]$
- Bước 5: $i = 2, j = 2 \rightarrow A[2] = 7 > B[2] = 5 \rightarrow$ lấy 5, $j++ \rightarrow result = [1, 2, 3, 4, 5]$
- Bước 6: $i = 2, j = 3 \rightarrow A[2] = 7 < B[3] = 8 \rightarrow$ lấy 7, $i++ \rightarrow result = [1, 2, 3, 4, 5, 7]$
- Bước 7: $i = 3, j = 3 \rightarrow A[3] = 9 > B[3] = 8 \rightarrow$ lấy 8, $j++ \rightarrow result = [1, 2, 3, 4, 5, 7, 8]$
- Bước 8: j hết $\rightarrow$ lấy $A[3] = 9 \rightarrow result = [1, 2, 3, 4, 5, 7, 8, 9]$

Code hoàn chỉnh

C++

```
vector<int> merge_sorted(vector<int> &A, vector<int> &B) {
    int i = 0, j = 0;
    vector<int> result;
    while (i < (int)A.size() && j < (int)B.size()) {
        if (A[i] <= B[j]) result.push_back(A[i++]);
        else result.push_back(B[j++]);
    }
    while (i < (int)A.size()) result.push_back(A[i++]);
    while (j < (int)B.size()) result.push_back(B[j++]);
    return result;
}
```

Python

```
def merge_sorted(A, B):
    i, j = 0, 0
    result = []
    while i < len(A) and j < len(B):
        if A[i] <= B[j]:
            result.append(A[i])
            i += 1
        else:
            result.append(B[j])
            j += 1
    while i < len(A):
        result.append(A[i])
        i += 1
    while j < len(B):
        result.append(B[j])
        j += 1
    return result
```

Độ phức tạp: $O(n + m)$ — mỗi phần tử được xét đúng 1 lần.

6. Bài toán mẫu hoàn chỉnh: Ferris Wheel (CSES 1090)

Đề bài: Có n trẻ em muốn lên đu quay. Trẻ thứ i nặng w_i kg. Mỗi cabin chứa tối đa 2 trẻ và chịu được tối đa x kg. Tìm số cabin ít nhất.

Input: 4 10 và 7 2 3 9. **Output:** 3.

Phân tích: Ghép trẻ nặng nhất với trẻ nhẹ nhất. Nếu không ghép được, trẻ nặng nhất ngồi một mình. Đây là bài hai con trỏ ngược chiều sau khi sắp xếp!

Mô phỏng với $w = [2, 3, 7, 9]$ (sau sort), $x = 10$:

- Bước 1: $l = 0, r = 3 \rightarrow 2 + 9 = 11 > 10 \rightarrow$ Trẻ 9kg ngồi một mình, $r -- \rightarrow cabins = 1$
- Bước 2: $l = 0, r = 2 \rightarrow 2 + 7 = 9 \leq 10 \rightarrow$ Ghép (2,7), $l ++, r -- \rightarrow cabins = 2$
- Bước 3: $l = 1, r = 1 \rightarrow l == r \rightarrow$ Trẻ 3kg ngồi một mình $\rightarrow cabins = 3$

C++

```
#include <bits/stdc++.h>
using namespace std;
int main() {
    int n, x;
    cin >> n >> x;
    vector<int> w(n);
    for (int &v : w) cin >> v;
    sort(w.begin(), w.end());
    int l = 0, r = n - 1, cabins = 0;
    while (l <= r) {
        if (l == r) { cabins++; break; }
        if (w[l] + w[r] <= x) { l++; r--; }
        else { r--; }
        cabins++;
    }
    cout << cabins << endl;
}
```

Python

```
n, x = map(int, input().split())
w = list(map(int, input().split()))
w.sort()
l, r = 0, len(w) - 1
cabins = 0
while l <= r:
    if l == r:
        cabins += 1
        break
    if w[l] + w[r] <= x:
        l += 1
        r -= 1
    else:
        r -= 1
    cabins += 1
```

```

    if w[l] + w[r] <= x:
        l += 1
        r -= 1
    else:
        r -= 1
    cabins += 1
print(cabins)

```

7. Sai lầm thường gặp và mẹo

Sai lầm 1: Quên xử lý cửa sổ rỗng

Khi $l > r$, $r - l + 1$ cho ra số âm. Đảm bảo l không vượt $r + 1$.

Sai lầm 2: Không duy trì bất biến (invariant)

C++

```

// SAI: cập nhật đáp án TRƯỚC khi kiểm tra
for (int r = 0; r < n; r++) {
    sum += a[r];
    ans = max(ans, r - l + 1);
    while (sum > S) sum -= a[l++];
}

// ĐÚNG: kiểm tra trước, cập nhật sau
for (int r = 0; r < n; r++) {
    sum += a[r];
    while (sum > S) sum -= a[l++];
    ans = max(ans, r - l + 1);
}

```

Python

```

# SAI: cập nhật đáp án TRƯỚC khi kiểm tra
for r in range(n):
    total += a[r]
    ans = max(ans, r - l + 1)
    while total > S:
        total -= a[l]
        l += 1

# ĐÚNG: kiểm tra trước, cập nhật sau
for r in range(n):
    total += a[r]
    while total > S:
        total -= a[l]

```

```
l += 1
ans = max(ans, r - l + 1)
```

Sai lầm 3: Dùng two pointers khi không có tính đơn điệu

Nếu mảng có số âm, "tổng đoạn $\leq S$ " không có tính đơn điệu và kỹ thuật này sẽ cho kết quả sai.

Mẹo 1: Đếm thay vì tìm max

Đếm số đoạn con thỏa mãn: với mỗi r , cộng $r - l + 1$ (tất cả đoạn $[l, r], [l + 1, r], \dots, [r, r]$).

Mẹo 2: Dùng map/set cho điều kiện phức tạp

Khi điều kiện là "tất cả phần tử khác nhau", " $\max - \min \leq k$ ", v.v., dùng `map`, `set`, hoặc `multiset` để theo dõi cửa sổ.

8. Tổng kết

DẠNG	KHI NÀO DÙNG	ĐỘ PHỨC TẠP
Cùng chiều (Sliding Window)	Tìm đoạn con thỏa mãn điều kiện đơn điệu	$O(n)$
Ngược chiều	Tìm cặp trên mảng đã sort	$O(n)$
Merge	Trộn/so sánh hai mảng đã sort	$O(n + m)$